



Instituto
Europeo
de Posgrado

Itinera: planificador de viajes personalizado mediante LLM Customization

Carlos Galán García

Docente: _____

Generative AI

2026



Solución al proyecto de aplicación

1. Enfoque de la solución

He planteado Itinera como un MVP público que convierte unas preferencias muy simples —destino, fechas, viajeros, dos bloques de presupuesto, intereses y observaciones— en un itinerario útil. La decisión principal ha sido no construir un chatbot genérico, sino una experiencia visual de viaje: primero se recogen los datos, después se genera una ruta por días y, al final, el usuario puede abaratarla, relajar el ritmo o adaptarla a lluvia. El límite de siete días solo existe para contener tiempo de respuesta y consumo de tokens durante la demostración; no es una limitación del planteamiento.

2. Arquitectura y personalización del LLM

La aplicación está desarrollada con Next.js en un único proyecto y no necesita base de datos. La ruta de servidor consulta Open-Meteo para geocodificación y previsión, y OpenTripMap para recuperar lugares e imágenes. Ese contexto, junto con las preferencias del usuario, se incorpora al prompt de un modelo preentrenado mediante la Responses API. Además, la herramienta de búsqueda web recupera información actual y da prioridad, por instrucción, a webs oficiales de turismo, atracciones y transporte y, en segundo lugar, a guías reconocidas. En la práctica, esta combinación funciona como RAG: primero recupero información externa y después se la paso al modelo para que redacte una respuesta fundamentada (OpenAI, 2026; OpenTripMap, s. f.; Open-Meteo, s. f.).

El prompt define el rol de planificador, fija el idioma, separa viaje y alojamiento del gasto diario, pide agrupar actividades por proximidad y prohíbe inventar disponibilidad o precios en tiempo real. También exige un JSON con una estructura cerrada. Esto evita que la interfaz dependa de texto libre y permite validar y representar siempre los mismos bloques: llegada, zona recomendada, presupuesto, días, consejos, advertencias y fuentes.

3. Calidad, seguridad y coste

He incluido varias capas de control. El servidor valida el destino y, si se configura, un código de acceso; limita la duración; transforma la salida a una estructura conocida; distingue visualmente entre información en vivo y modo demostración; y muestra avisos para confirmar horarios, tarifas y reservas. Si falla una API o no hay claves, la aplicación genera un plan coherente con datos de demostración en vez de romperse. Para ciudades poco documentadas conserva el destino, avisa de la incertidumbre y no asigna un país inventado. Las claves solo viven en variables de entorno del servidor. Además, el proveedor puede tener un tope de gasto, por lo que una prueba docente debería costar solo unos céntimos.

4. Fine-tuning y alcance

Como el entrenamiento de fine-tuning no está habilitado en mi organización de OpenAI, he preparado un experimento reproducible en Colab con Qwen3-0.6B y LoRA. El notebook genera 48 conversaciones de ejemplo, separa entrenamiento y validación, entrena adaptadores pequeños y compara una petición antes y después. El ajuste busca aprender el tono y el formato de Itinera, no datos actuales: precios, clima y horarios deben seguir llegando del RAG. Esta separación me parece importante porque reduce alucinaciones y hace que el experimento sea barato y explicable (Hugging Face, 2026a, 2026b). No he añadido GAN ni aprendizaje por refuerzo porque el enunciado los marca como opcionales y no aportarían valor proporcional en un MVP de un día.



Aplicación Práctica del Conocimiento

Aplicación práctica

La utilidad más clara es transformar información dispersa en una primera propuesta accionable. En vez de abrir diez pestañas para entender una ciudad, el usuario recibe una ruta organizada, sabe qué parte del presupuesto está estimada y puede ir directamente a las fuentes antes de reservar. No sustituye a una agencia ni garantiza disponibilidad; reduce el trabajo inicial y ayuda a formular mejores decisiones.

Uso académico y profesional

Este proyecto me ha servido para unir conceptos que por separado parecen abstractos. El LLM aporta redacción y capacidad de síntesis; el prompting convierte una intención abierta en una salida controlada; el RAG introduce datos externos; LoRA muestra cómo especializar comportamiento sin reentrenar todo el modelo; y la interfaz convierte esas piezas en un producto que una persona puede probar. En un contexto profesional aplicaría el mismo patrón a asistentes de atención, comparadores o herramientas internas: datos verificables por un lado y generación por otro.

Decisiones y resolución de problemas

Simplicidad. He evitado LangChain, agentes múltiples y una base de datos porque para esta entrega añaden superficie de fallo sin mejorar el resultado visible. Un único repositorio hace más fácil mantener y desplegar el proyecto.

Fiabilidad. La búsqueda web nunca se trata como verdad absoluta: el prompt prioriza fuentes reconocibles, la respuesta conserva enlaces y los precios se presentan como orientativos. Cuando faltan datos, se declara en lugar de completarlos con una certeza falsa.

Experiencia. El resultado se guarda en el navegador, puede imprimirse, cambia entre español e inglés y ofrece tema claro u oscuro. Los tres refinamientos se ejecutan localmente para que sean inmediatos y no consuman otra llamada al modelo.

Evolución. La siguiente iteración incorporaría autenticación real, persistencia compartida, comparación de vuelos y alojamientos mediante proveedores autorizados y una evaluación sistemática de itinerarios con más revisores. Para este MVP he preferido que cada función presente sea demostrable.

Conclusión

El resultado cubre los ocho requisitos del caso: entrada de preferencias, modelo preentrenado, prompting, RAG, experimento de fine-tuning, control de calidad, interfaz y modificaciones. La parte opcional de GAN/RL queda razonadamente fuera. Sobre todo, queda una aplicación que se puede enseñar, probar y seguir desarrollando, no solo un ejercicio aislado en Colab.



Instituto
Europeo
de Posgrado

Referencias

Hugging Face. (2026a). PEFT: LoRA. https://huggingface.co/docs/peft/package_reference/lora

Hugging Face. (2026b). TRL: SFT Trainer. https://huggingface.co/docs/trl/sft_trainer

OpenAI. (2026). Web search tool. <https://developers.openai.com/api/docs/guides/tools-web-search>

Open-Meteo. (s. f.). Weather Forecast API. <https://open-meteo.com/en/docs>

OpenTripMap. (s. f.). API documentation. <https://dev.opentripmap.org/>